



M.Tech Software Engineering, SCORE

Software Testing Lab

FINAL PROJECT REPORT

Hospital Management System

Software Testing using Selenium WebDriver

Name	Register No.
ARUN C	23MIS0445
SAI CHANDAN K S	23MIS0115

GitHub repository: <https://github.com/kssaichandan/hospital-managment->

Submitted: October 2026

1. Abstract

The **Hospital Management System (HMS)** is a web application that brings the daily work of a hospital into one place: patient registration, doctor management, appointment scheduling, medical records and billing. In many hospitals this information is still kept on paper or in separate files, which leads to duplicate data, mistakes, slow searching and delays for patients.

Because this is a **Software Testing** project, the main focus is on verifying that the system works correctly. We tested the application automatically with **Selenium WebDriver**, which controls a real Google Chrome browser in the same way a user would: it opens pages, types into forms, clicks buttons and reads the result on the screen. Every Selenium test compares the actual result with the expected result and reports **PASS** or **FAIL**.

We designed **37 Selenium test cases**: the 9 test cases from our project document plus extra test cases for every module. They cover **positive** tests (valid input), **negative** tests (wrong or missing input), **boundary and edge** tests (values exactly at the limits) and **security** tests (SQL injection, script injection and opening pages without logging in). Together with 85 PyTest tests at the unit, integration and functional level, the project has **122 automated tests, and all of them pass**.

Objectives

- ✓ Develop a working Hospital Management System with six modules.
- ✓ Automate the testing of the web application with Selenium WebDriver.
- ✓ Verify the correctness of every module for valid and invalid inputs.
- ✓ Check the limits of every input with boundary value analysis.
- ✓ Check basic security: SQL injection, script injection and protected pages.
- ✓ Reduce repetitive manual testing so the full test suite can be run again after every change (regression testing).

Key results

37

Selenium test cases

122

automated tests in total

100%

pass rate (0 failed)

6

modules tested

Keywords: Software testing, Selenium WebDriver, test automation, black-box testing, positive and negative testing, boundary value analysis, regression testing, Python, PyTest.

2. Tool Used: Selenium WebDriver

Selenium is a free, open-source tool for automating web browsers. It is mainly used to test web applications. With Selenium WebDriver we write a program that controls a real browser: it opens a web page, finds elements such as text boxes and buttons, types text, clicks, and reads what is shown on the page. Selenium supports Chrome, Firefox, Edge and Safari, and test code can be written in Java, Python, C#, JavaScript and Ruby. **We used Python.**

Components of Selenium

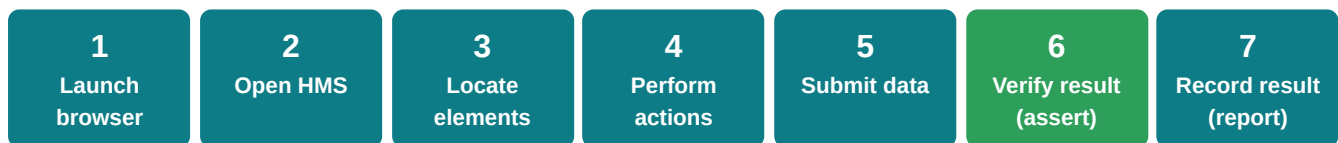
Component	What it does	Used by us
Selenium WebDriver	Controls the browser from code: open, find, type, click, read	Yes
Selenium IDE	Browser extension that records and replays user actions	No
Selenium Grid	Runs the same tests on many browsers and computers at once	No

How Selenium WebDriver works



Selenium 4 includes *Selenium Manager*, which downloads the correct ChromeDriver automatically.

Selenium workflow in our project



Why Selenium, and what kind of testing it is

- **Automation testing:** the same tests run again in minutes after every change, always in the same way.
- **Black-box testing:** Selenium tests the system only through the user interface, without looking at the source code. It is used for **functional, system (end-to-end) and regression testing**.
- **Free and open-source**, works with many browsers and programming languages.
- **Locators:** Selenium finds elements by ID, name, class name, tag name, link text, CSS selector or XPath. We used **By.ID** because ids are unique and do not change when the design changes.

Other tools and technologies

Python 3 · Flask language and web framework	SQLite database for all hospital data
HTML · CSS user interface of the pages	PyTest · pytest-html runs the tests, HTML report
Chrome · ChromeDriver browser controlled by Selenium	Git · GitHub version control, source code

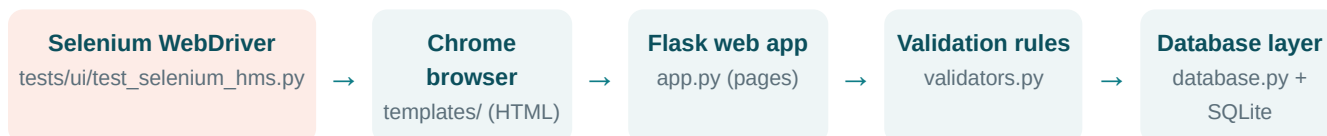
3. Project Description

The Hospital Management System is used by hospital staff after logging in. The administrator can manage the whole hospital from one dashboard. Every input is checked by validation rules before it is saved, and wrong input is rejected with a clear red message.

Modules

Module	Features
Login	Username and password check, error message for wrong login, logout; every page is protected until the user logs in.
Patient Management	Add, view, update and delete patients; search by Patient ID or name. Phone must be 10 digits, age 0–120.
Doctor Management	Add, update and search doctors; mark a doctor Available or Not Available.
Appointments	Select patient, doctor, date and time; book or cancel. A doctor or a patient cannot be booked twice at the same time, and dates in the past are rejected.
Medical Records	Diagnosis, treatment and prescription for each visit; full history of each patient.
Billing	Consultation charge + treatment charge; the total is calculated automatically; Paid / Unpaid status.

System architecture



Project structure

app.py	Flask web application: all pages and routes
database.py	SQLite database: create, read, update, delete
validators.py	Input validation rules
templates/, static/	HTML pages and styling
tests/test_validators.py	Unit tests
tests/test_database.py	Integration tests
tests/test_routes.py	Functional tests (pages, without a browser)
tests/ui/test_selenium_hms.py	37 Selenium WebDriver test cases
run.py	Live demo: Selenium tests the running app step by step
login.py	Simple first Selenium test (login)
report.py	Runs all 122 tests and opens the HTML test report
docs/	Screenshots, HTML test report, demo guide

Source code and how to run

GitHub: <https://github.com/kssaichandan/hospital-managment->

```
pip install -r requirements.txt
python app.py           # start the HMS at http://127.0.0.1:5000 (login: admin / admin123)
python run.py           # Selenium tests the running app, step by step, in Chrome
python report.py        # run all 122 tests and open the HTML test report
```

4. Application Screenshots

Screenshots of the running Hospital Management System.

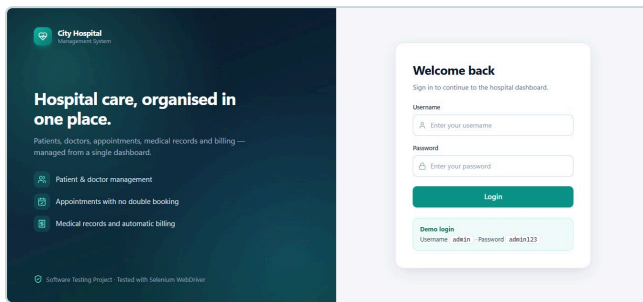


Figure 1: Login page

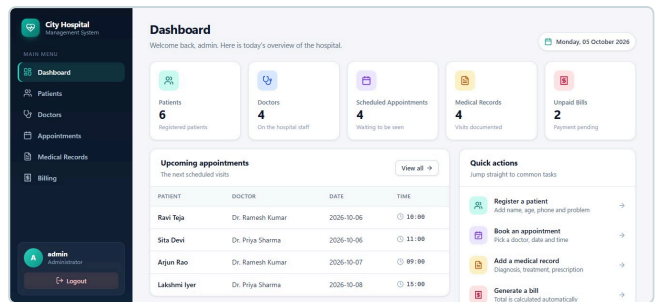


Figure 2: Dashboard

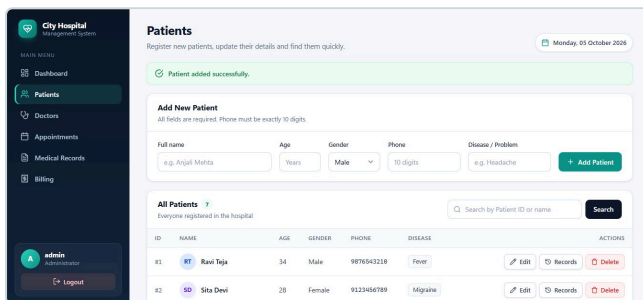


Figure 3: Patient management

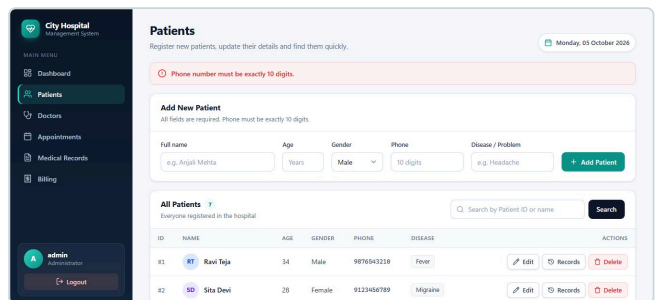


Figure 4: Validation: wrong input shows a red message

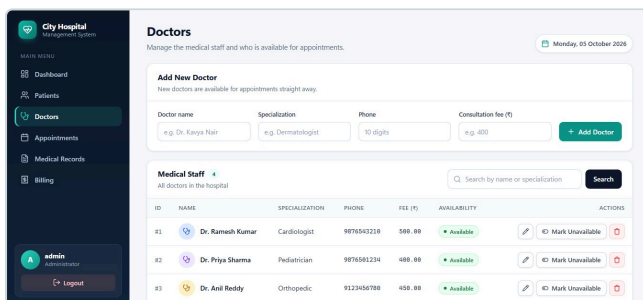


Figure 5: Doctor management and availability

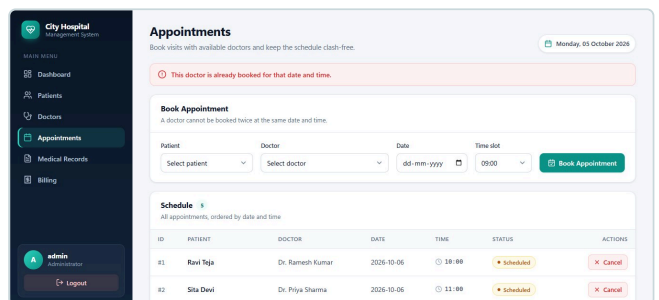


Figure 6: Appointments: double booking is blocked

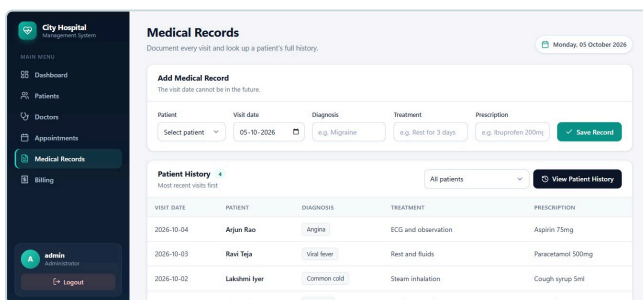


Figure 7: Medical records

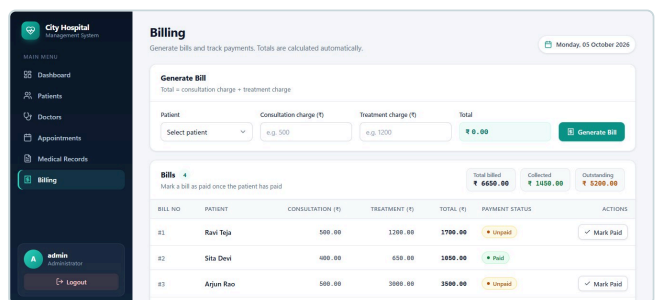


Figure 8: Billing: total calculated automatically

5. Implementation

All input is checked by small validation functions in `validators.py`. If the input is wrong, the function raises an error with a friendly message, the web page shows it in red, and nothing is saved.

```
validators.py - validation rules (age and phone)

def validate_age(age):
    try:
        age = int(age)
    except (TypeError, ValueError):
        raise ValueError("Age must be a number.")
    if age < 0 or age > 120:
        raise ValueError("Age must be between 0 and 120.")
    return age

def validate_phone(phone):
    phone = (phone or "").strip()
    # isascii() makes sure only the normal digits 0-9 are accepted
    if len(phone) != 10 or not (phone.isascii() and phone.isdigit()):
        raise ValueError("Phone number must be exactly 10 digits.")
    return phone
```

The business rules of the appointment module are in `database.py`. Before an appointment is saved, the system checks that the patient and doctor exist, that the doctor is available, that the date is not in the past, and that neither the doctor nor the patient already has an appointment at that time.

```
database.py - booking an appointment with its business rules

def book_appointment(self, patient_id, doctor_id, date, time):
    if not self.get_patient(patient_id):
        raise ValueError("Please select a valid patient.")
    doctor = self.get_doctor(doctor_id)
    if not doctor:
        raise ValueError("Please select a valid doctor.")
    if not doctor["available"]:
        raise ValueError("This doctor is not available right now.")
    date = v.validate_appointment_date(date)
    time = v.validate_time_slot(time)
    # Business rule: a doctor cannot have two appointments at the same time
    clash = self._query(
        "SELECT id FROM appointments WHERE doctor_id = ? AND date = ? AND time = ? "
        "AND status = 'Scheduled'",
        (doctor_id, date, time),
    )
    if clash:
        raise ValueError("This doctor is already booked for that date and time.")
    # Business rule: a patient cannot be in two appointments at the same time
    clash = self._query(
        "SELECT id FROM appointments WHERE patient_id = ? AND date = ? AND time = ? "
        "AND status = 'Scheduled'",
        (patient_id, date, time),
    )
    if clash:
        raise ValueError("This patient already has an appointment at that date and time.")
    return self._execute(
        "INSERT INTO appointments (patient_id, doctor_id, date, time) VALUES (?, ?, ?, ?)",
        (patient_id, doctor_id, date, time),
    )
```

6. Testing Approach

We tested the project at four levels. The Selenium tests are the main part of the project; the PyTest tests check the code underneath without a browser.

Level	File	What it checks	Tests
Unit	tests/test_validators.py	Each validation rule on its own	39
Integration	tests/test_database.py	Validation and database together	28
Functional	tests/test_routes.py	Every page through HTTP requests, without a browser	18
System / UI	tests/ui/test_selenium_hms.py	The full application in Chrome with Selenium	37
Total			122

The 37 Selenium test cases are of four types: **10 positive** (valid input must work), **17 negative** (wrong input must show an error and nothing may be saved), **7 boundary / edge** (for example age 0 and 120 accepted, 121 rejected) and **3 security** (SQL injection, script injection, pages without login).

Example Selenium test: TC02 (wrong password)

```
tests/ui/test_selenium_hms.py

def login(driver, base_url, username="admin", password="admin123"):
    driver.get(base_url)
    driver.find_element(By.ID, "username").send_keys(username)
    driver.find_element(By.ID, "password").send_keys(password)
    click_and_wait(driver, "login")

def test_TC02_login_with_invalid_details(driver, live_server):
    login(driver, live_server, "admin", "wrongpassword")
    assert "Invalid username or password" in error_message(driver)
    assert "/dashboard" not in driver.current_url
```

`driver.get()` opens the page, `find_element(By.ID, ...)` finds a text box by its id, `send_keys()` types into it, `click_and_wait()` clicks and waits for the next page, and the `assert` lines check the expected result: if they are true the test passes, otherwise it fails.

Example boundary test: TC21 (age 121)

```
tests/ui/test_selenium_hms.py

def test_TC21_patient_age_121_is_rejected(driver, live_server): # Boundary (just above 120)
    login(driver, live_server)
    add_patient(driver, live_server, name="Ravi Teja", age="121")
    assert "Age must be between 0 and 120" in error_message(driver)
    assert "No patients found" in table_text(driver, "patients-table")
```

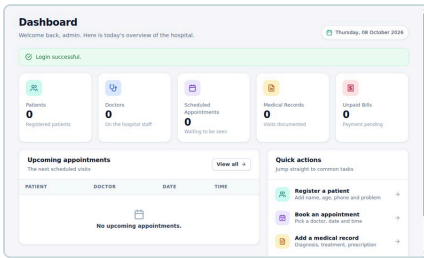
7. Selenium Test Cases

TC01–TC09 are the test cases from our project document; TC10–TC37 cover the other modules and the negative, boundary and security cases.

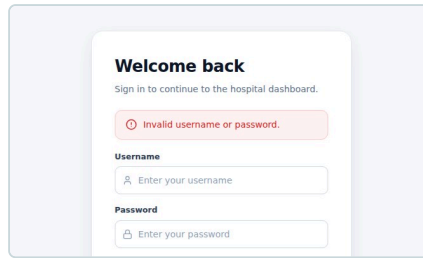
ID	Type	Test case	Input	Expected result	Result
TC01	Positive	Login with valid details	Correct username/password	Login successful	PASS
TC02	Negative	Login with invalid details	Wrong password	Error message displayed	PASS
TC03	Positive	Add patient	Valid patient details	Patient added	PASS
TC04	Negative	Empty patient form	Empty fields	Validation message	PASS
TC05	Positive	Book appointment	Valid doctor/date	Appointment booked	PASS
TC06	Positive	Search patient	Patient ID	Patient details displayed	PASS
TC07	Positive	Update patient	Modified details	Details updated	PASS
TC08	Positive	Delete patient	Existing patient	Patient removed	PASS
TC09	Positive	Logout	Click Logout	User returned to login page	PASS
TC10	Positive	Doctor availability	Mark doctor unavailable	Shows "Not Available"	PASS
TC11	Negative	Double booking	Same doctor, date and time	Error "already booked"	PASS
TC12	Positive	Medical record	Diagnosis, treatment, prescription	Record saved	PASS
TC13	Positive	Billing total	500 + 1200	Total = 1700.00	PASS
TC14	Negative	Login with wrong username	Username doctor	Error message displayed	PASS
TC15	Negative	Login with empty fields	Empty username/password	Error message displayed	PASS
TC16	Security	SQL injection in login	admin' -- / ' OR '1'=1	Login refused	PASS
TC17	Security	Page without login	Open /patients directly	Sent to login page	PASS
TC18	Negative	Phone too short	Phone 12345	Error, not saved	PASS
TC19	Boundary	Phone one digit too long	Phone with 11 digits	Error, not saved	PASS
TC20	Negative	Name with numbers	Name Ravi123	Error, not saved	PASS
TC21	Boundary	Age just above limit	Age 121	Error, not saved	PASS
TC22	Boundary	Age at the limits	Age 0 and age 120	Both accepted	PASS
TC23	Negative	Search unknown patient	Patient ID 99999	"No patients found"	PASS
TC24	Negative	Invalid patient update	Phone 123	Error, old data kept	PASS
TC25	Security	Script injection (XSS)	<script> in disease	Shown as text, not run	PASS
TC26	Boundary	Doctor fee 0	Fee 0	Error, not saved	PASS
TC27	Negative	Search unknown doctor	Veterinarian	"No doctors found"	PASS
TC28	Negative	Appointment in the past	Yesterday's date	Error, not booked	PASS
TC29	Negative	Appointment without patient	No patient selected	Error, not booked	PASS
TC30	Negative	Unavailable doctor	Doctor marked unavailable	Error, not booked	PASS
TC31	Negative	Patient double booking	Same patient, 2 doctors, same time	Error, not booked	PASS
TC32	Edge	Re-book cancelled slot	Cancel, then book again	Booked again	PASS
TC33	Negative	Record without diagnosis	Empty diagnosis	Error, not saved	PASS
TC34	Negative	Record with future date	Tomorrow's date	Error, not saved	PASS
TC35	Boundary	Bill total 0	0 + 0	Error, not saved	PASS
TC36	Negative	Negative charge	-100 + 500	Error, not saved	PASS
TC37	Edge	Decimal amounts	499.50 + 0.25	Total = 499.75	PASS

8. Screenshots of All Selenium Tests

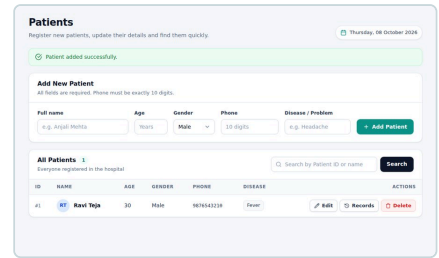
The screen at the end of every test case, saved automatically by the test. Login-page tests show the login form; the other tests show the page without the menu.



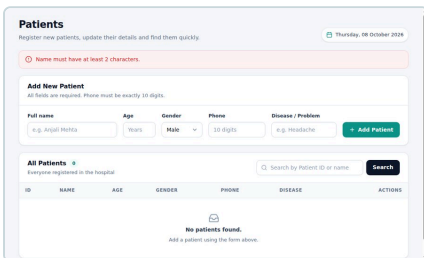
TC01 Login with valid details **PASS**



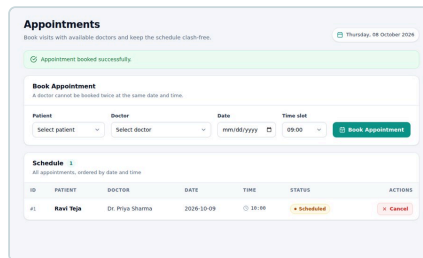
TC02 Login with invalid details **PASS**



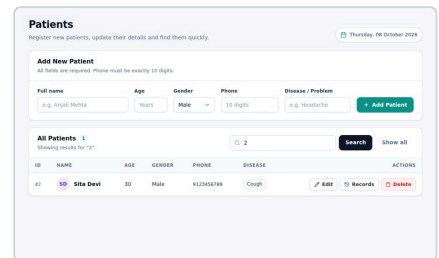
TC03 Add patient **PASS**



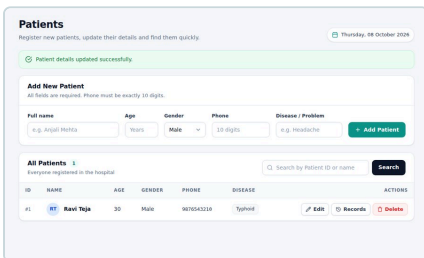
TC04 Empty patient form **PASS**



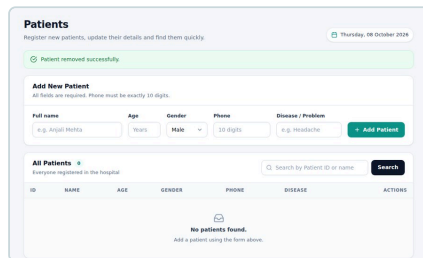
TC05 Book appointment **PASS**



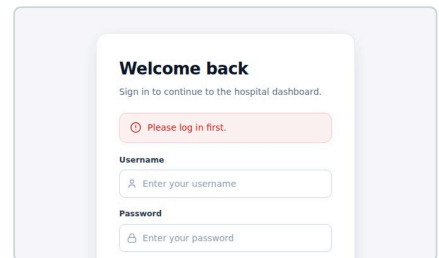
TC06 Search patient **PASS**



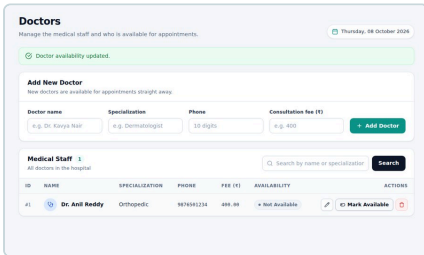
TC07 Update patient **PASS**



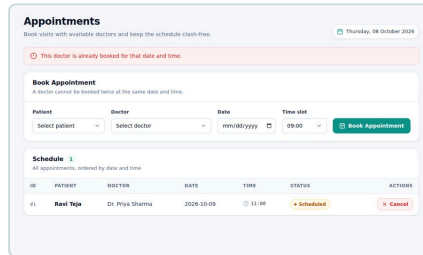
TC08 Delete patient **PASS**



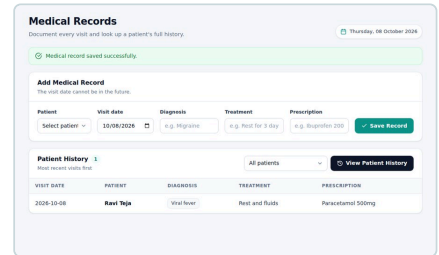
TC09 Logout **PASS**



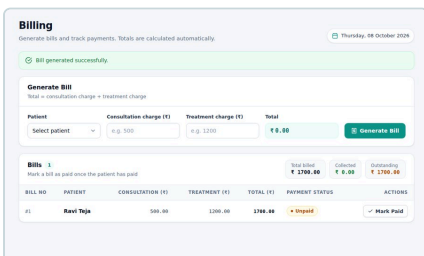
TC10 Doctor availability **PASS**



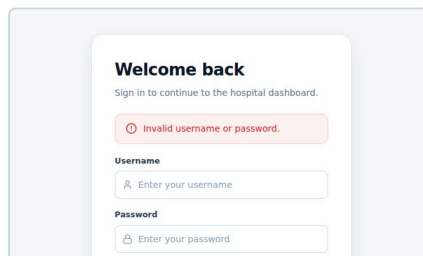
TC11 Double booking **PASS**



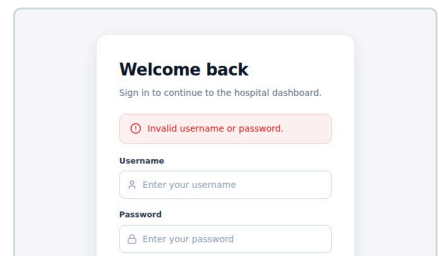
TC12 Medical record **PASS**



TC13 Billing total **PASS**

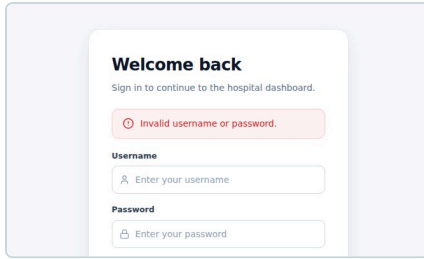


TC14 Login with wrong username **PASS**

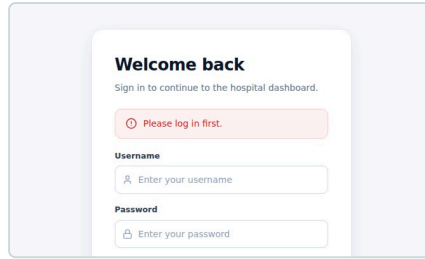


TC15 Login with empty fields **PASS**

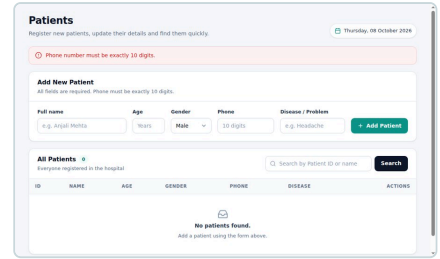
8. Screenshots of All Selenium Tests (continued)



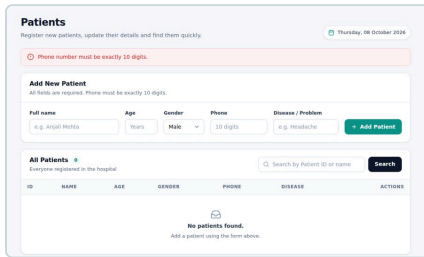
TC16 SQL injection in login **PASS**



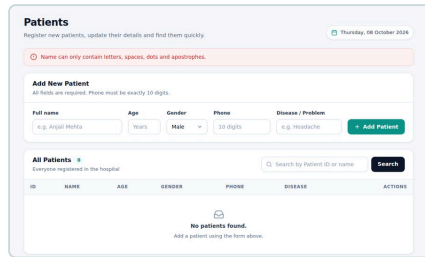
TC17 Page without login **PASS**



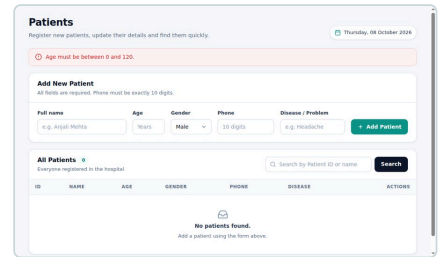
TC18 Phone too short **PASS**



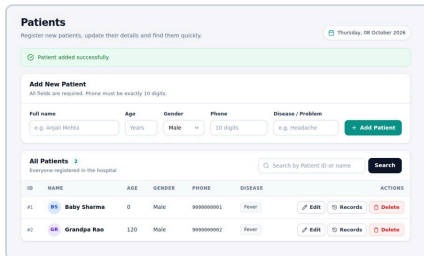
TC19 Phone one digit too long **PASS**



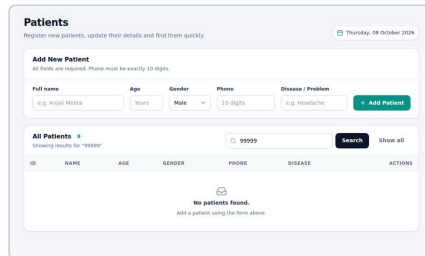
TC20 Name with numbers **PASS**



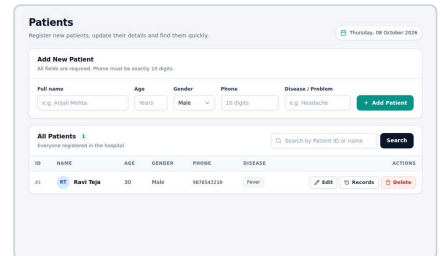
TC21 Age just above limit **PASS**



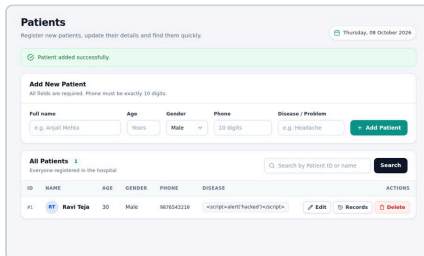
TC22 Age at the limits **PASS**



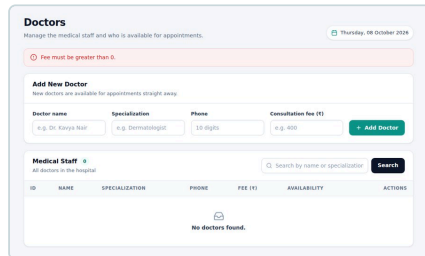
TC23 Search unknown patient **PASS**



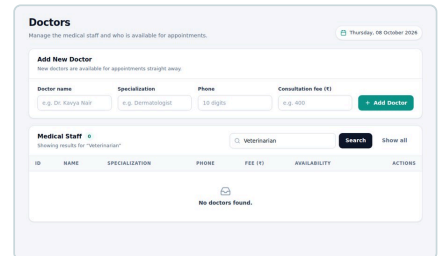
TC24 Invalid patient update **PASS**



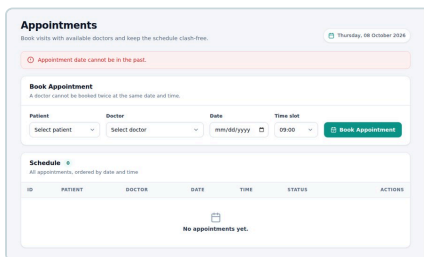
TC25 Script injection (XSS) **PASS**



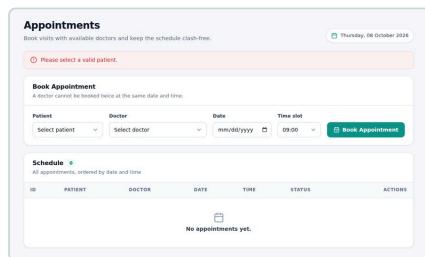
TC26 Doctor fee 0 **PASS**



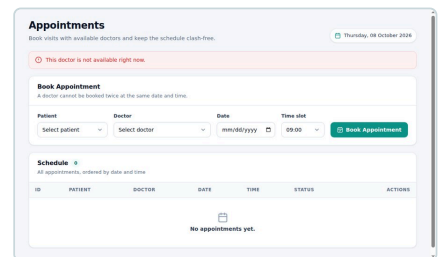
TC27 Search unknown doctor **PASS**



TC28 Appointment in the past **PASS**



TC29 Appointment without patient **PASS**



TC30 Unavailable doctor **PASS**

8. Screenshots of All Selenium Tests (continued)

Appointments
Book visits with available doctors and keep the schedule clash-free.

This patient already has an appointment at that date and time.

Book Appointment
A doctor cannot be booked twice at the same date and time.

Patient: Select patient | Doctor: Select doctor | Date: mm/dd/yyyy | Time slot: 09:00 | Book Appointment

Schedule
All appointments, ordered by date and time.

ID	PATIENT	DOCTOR	DATE	TIME	STATUS	ACTIONS
#1	Ravi Teja	Dr. Priya Sharma	2026-10-09	10:00	Scheduled	Cancel

TC31 Patient double booking **PASS**

Appointments
Book visits with available doctors and keep the schedule clash-free.

Appointment booked successfully.

Book Appointment
A doctor cannot be booked twice at the same date and time.

Patient: Select patient | Doctor: Select doctor | Date: mm/dd/yyyy | Time slot: 09:00 | Book Appointment

Schedule
All appointments, ordered by date and time.

ID	PATIENT	DOCTOR	DATE	TIME	STATUS	ACTIONS
#1	Ravi Teja	Dr. Priya Sharma	2026-10-09	10:00	Cancelled	Cancel
#2	Ravi Teja	Dr. Priya Sharma	2026-10-09	10:00	Scheduled	Cancel

TC32 Re-book cancelled slot **PASS**

Medical Records
Document every visit and link up a patient's full history.

Diagnosis is required.

Add Medical Record
The visit date cannot be in the future.

Patient: Select patient | Visit date: 10/08/2026 | Diagnosis: e.g. Migraine | Treatment: e.g. Rest for 3 days | Prescription: e.g. Ibuprofen 200 | Save Record

Patient History
Most recent visits first.

VISIT DATE	PATIENT	DIAGNOSIS	TREATMENT	PRESCRIPTION
------------	---------	-----------	-----------	--------------

No medical records found.

TC33 Record without diagnosis **PASS**

Medical Records
Document every visit and link up a patient's full history.

Visit date cannot be in the future.

Add Medical Record
The visit date cannot be in the future.

Patient: Select patient | Visit date: 10/08/2026 | Diagnosis: e.g. Migraine | Treatment: e.g. Rest for 3 days | Prescription: e.g. Ibuprofen 200 | Save Record

Patient History
Most recent visits first.

VISIT DATE	PATIENT	DIAGNOSIS	TREATMENT	PRESCRIPTION
------------	---------	-----------	-----------	--------------

No medical records found.

TC34 Record with future date **PASS**

Billing
Generate bills and track payments. Totals are calculated automatically.

Total amount must be greater than 0.

Generate Bill
Total = consultation charge + treatment charge

Patient: Select patient | Consultation charge (₹): e.g. 500 | Treatment charge (₹): e.g. 1200 | Total: ₹ 0.00 | Generate Bill

Bills
Mark a bill as paid once the patient has paid.

BILL NO	PATIENT	CONSULTATION (₹)	TREATMENT (₹)	TOTAL (₹)	PAYMENT STATUS	ACTIONS
---------	---------	------------------	---------------	-----------	----------------	---------

No bills yet.

TC35 Bill total 0 **PASS**

Billing
Generate bills and track payments. Totals are calculated automatically.

Consultation charge cannot be negative.

Generate Bill
Total = consultation charge + treatment charge

Patient: Select patient | Consultation charge (₹): e.g. 500 | Treatment charge (₹): e.g. 1200 | Total: ₹ 0.00 | Generate Bill

Bills
Mark a bill as paid once the patient has paid.

BILL NO	PATIENT	CONSULTATION (₹)	TREATMENT (₹)	TOTAL (₹)	PAYMENT STATUS	ACTIONS
---------	---------	------------------	---------------	-----------	----------------	---------

No bills yet.

TC36 Negative charge **PASS**

Billing
Generate bills and track payments. Totals are calculated automatically.

Bill generated successfully.

Generate Bill
Total = consultation charge + treatment charge

Patient: Select patient | Consultation charge (₹): e.g. 500 | Treatment charge (₹): e.g. 1200 | Total: ₹ 0.00 | Generate Bill

Bills
Mark a bill as paid once the patient has paid.

BILL NO	PATIENT	CONSULTATION (₹)	TREATMENT (₹)	TOTAL (₹)	PAYMENT STATUS	ACTIONS
#1	Ravi Teja	499.50	0.25	499.75	Unpaid	Mark Paid

TC37 Decimal amounts **PASS**

Live demo with run.py

City Hospital Management System

Patients
Register new patients, update their details and find them quickly.

Add New Patient
All fields are required. Phone must be exactly 10 digits.

Full name: Ravi Teja | Age: 30 | Gender: Male | Phone: 9876543210 | Disease / Problem: e.g. Headache | Add Patient

All Patients
Everyone registered in the hospital.

ID	NAME	AGE	GENDER	PHONE	DISEASE	ACTIONS
#1	Ravi Teja	34	Male	9876543210	Fever	Edit Records Delete
#2	Sita Devi	28	Female	9123456789	Migraine	Edit Records Delete
#3	SELENIUM - TC33 Add patient	Type "9876543210" into #patient-phone				Edit Records Delete

Selenium typing into the patient form (orange box = element found by its id)

SELENIUM WEBDRIVER - LIVE TEST RUN
City Hospital HMS — Test Results

37 Test cases run | 37 Passed | 0 Failed | 91 s Total time

ID	TYPE	TEST CASE	INPUT	EXPECTED RESULT	RESULT	TIME
TC01	Positive	Login with valid details	Correct username/password	Login successful	PASS	0.9 s
TC02	Negative	Login with invalid details	Wrong password	Error message displayed	PASS	1.1 s
TC03	Positive	Add patient	Valid patient details	Patient added	PASS	2.0 s
TC04	Negative	Empty patient form	Empty fields	Validation message	PASS	0.7 s
TC05	Positive	Book appointment	Valid doctor/date	Appointment booked	PASS	4.3 s

Results page at the end of the live demo: 37 passed, 0 failed

9. Test Results

122

automated tests

122

passed

0

failed

6 / 6

planted bugs caught

Summary

122 tests took 00:01:32.

(Un)check the boxes to filter the results.

☐ 0 Failed, ☒ 122 Passed, ☐ 0 Skipped, ☐ 0 Expected failures, ☐ 0 Unexpected passes, ☐ 0 Errors, ☐ 0 Reruns ☐ 0 Retried,

[Show all details](#) / [Hide all details](#)

Result	Test	Duration	Links
Passed	tests/ui/test_selenium_hms.py::test_TC01_login_with_valid_details	00:00:02	
Passed	tests/ui/test_selenium_hms.py::test_TC02_login_with_invalid_details	00:00:01	
Passed	tests/ui/test_selenium_hms.py::test_TC03_add_patient_with_valid_details	00:00:02	
Passed	tests/ui/test_selenium_hms.py::test_TC04_empty_patient_form_shows_validation_message	00:00:01	
Passed	tests/ui/test_selenium_hms.py::test_TC05_book_appointment_with_valid_doctor_and_date	00:00:03	
Passed	tests/ui/test_selenium_hms.py::test_TC06_search_patient_by_id	00:00:03	
Passed	tests/ui/test_selenium_hms.py::test_TC07_update_patient_details	00:00:02	
Passed	tests/ui/test_selenium_hms.py::test_TC08_delete_existing_patient	00:00:02	
Passed	tests/ui/test_selenium_hms.py::test_TC09_logout_returns_to_login_page	00:00:01	
Passed	tests/ui/test_selenium_hms.py::test_TC10_add_doctor_and_mark_unavailable	00:00:02	
Passed	tests/ui/test_selenium_hms.py::test_TC11_doctor_cannot_be_double_booked	00:00:05	
Passed	tests/ui/test_selenium_hms.py::test_TC12_add_medical_record	00:00:03	
Passed	tests/ui/test_selenium_hms.py::test_TC13_generate_bill_calculates_total	00:00:03	

HTML test report created by pytest-html (python report.py): 122 tests, 122 passed, 0 failed

Mutation testing: to prove that our tests really detect bugs, we put 6 bugs into a copy of the application on purpose (for example: allow age 121, allow an 11-digit phone number, make the login open to SQL injection, allow a bill of 0). Every one of them made its test fail, so the tests do check the system.

10. Conclusion

We developed a working Hospital Management System with six modules and tested it with Selenium WebDriver. All 9 test cases from our project document and the 28 additional test cases pass, for valid and invalid inputs, at the limits of every input, and for basic security attacks. Including the PyTest tests, the project has 122 automated tests and all of them pass. Because the tests are automated, the complete suite can be run again in under two minutes after every change, which makes regression testing fast and reliable. The project shows how automated testing with Selenium improves the quality of a real web application.

References

1. Selenium documentation – <https://www.selenium.dev/documentation/>
2. PyTest documentation – <https://docs.pytest.org/>
3. Flask documentation – <https://flask.palletsprojects.com/>
4. Project source code – <https://github.com/kssaichandan/hospital-managment->